

ADVANCE WEB DEVELOPMENT FRAMEWORK (ITUE301) — SEMESTER 5

COMPLETE LABORATORY MANUAL & EXECUTION HANDBOOK
Comprehensive Guide for Practicals 1 to 13 With Standalone Run Commands

Student Name:	Keyur Rana	Academic Year:	2026 - Semester 5
Student ID:	D25DCE176	Subject:	AWDF (ITUE301)
Branch:	B.Tech Information Technology	Platform:	Node.js 24 • React 18/19 • Vite • MongoDB
GitHub Repository:	https://github.com/Ranakeyur-31525/awdf	All Practicals:	1 to 13 Verified & Standalone

Course Architecture & Highlights:

- **Independent Standalone Execution:** Every practical (1 to 13) has its own dedicated directory, conflict-free localhost port, and one-command run script.
- **Merged Full-Stack Project:** Practical 6 maintains the fully integrated decoupled architecture connecting Express, Mongoose, and React.
- **Interactive Browser Dashboards:** Pure backend practicals (P4, P5, P9, P10, P12) serve responsive HTML interfaces for instant evaluation.
- **Standardized 10-Page PDFs:** Every practical is accompanied by an individual 10-page syllabus-compliant laboratory report PDF.

Master Port & Command Execution Matrix

The following table provides the complete directory path, technology stack, dedicated localhost URL, and root run command for all 13 practicals and the Master Showcase portal.

P#	Practical Title	Type	Localhost URL	Run Command
01	Student Portfolio SPA	Frontend (React+Vite)	http://localhost:5171	<code>npm run p1</code>
02	State Management & Routing	Frontend (Router v6)	http://localhost:5172	<code>npm run p2</code>
03	GitHub API Integration	Frontend (REST Client)	http://localhost:5173	<code>npm run p3</code>
04	Express REST API (In-Memory)	Server + Web UI	http://localhost:4004	<code>npm run p4</code>
05	MongoDB & Mongoose Schema	ODM API + Web UI	http://localhost:4005	<code>npm run p5</code>
06	Full-Stack App (MERGED)	Full Stack Decoupled	http://localhost:5006 http://localhost:5176	<code>npm run p6:backend</code> <code>npm run p6:frontend</code>
07	JWT Auth & Middleware	Auth Server + React	http://localhost:5001 http://localhost:5177	<code>npm run p7:backend</code> <code>npm run p7:frontend</code>
08	React Lazy Loading & Suspense	Frontend (Code Split)	http://localhost:5178	<code>npm run p8</code>
09	In-Memory Caching (node-cache)	Cache API + Live UI	http://localhost:5002	<code>npm run p9</code>
10	Event-Driven Architecture	EventEmitter + UI	http://localhost:5003	<code>npm run p10</code>
11	Docker Containerization	3-Tier Compose App	http://localhost:5011 http://localhost:5181	<code>npm run p11:backend</code> <code>npm run p11:frontend</code>
12	CI/CD Pipeline & Auto Tests	CI Visualizer Server	http://localhost:5012	<code>npm run p12</code>
13	AI API Integration (Gemini)	Proxy Server + React	http://localhost:5004 http://localhost:5183	<code>npm run p13:backend</code> <code>npm run p13:frontend</code>
Master	Master Showcase Portal	Unified 1-13 Portal	http://localhost:5000 http://localhost:5180	<code>npm run dev:backend</code> <code>npm run dev:frontend</code>

Global System Prerequisites:

1. **Node.js:** v18.0.0 or higher (v24.x tested & verified).
2. **MongoDB:** Running locally on default port 27017 (e.g. `mongodb://127.0.0.1:27017`).
3. **Root Dependencies:** Run `npm install` once at root to provide shared runtime packages.

Practical 1: Student Portfolio (React + Vite)

Directory: Practical_01_Student_Portfolio | **Localhost URL:** <http://localhost:5171>

Tech Stack: React 19, Vite, Lucide Icons, Vanilla CSS

Execution & Run Command:

```
npm run p1
```

Module Overview & Architecture:

Modular single-page developer portfolio showcasing student information, academic credentials, technical skill badges, interactive project cards with live links, and an interactive 2-page PDF resume viewer with fullscreen zoom and download capability.

Key Endpoints / Routes: Frontend SPA. Static route '/' serving index.html and Vite assets.

Verification & Test Method: Browser verification on <http://localhost:5171> - check theme toggle, responsive layout, resume download button.

Practical 2: State Management & Multi-Page Routing

Directory: Practical_02_Routing_State | **Localhost URL:** <http://localhost:5172>

Tech Stack: React Router v6/v7, useState, useEffect, Lucide Icons

Execution & Run Command:

```
npm run p2
```

Module Overview & Architecture:

Client-side routing architecture with React Router v6. Implements multi-page navigation across Home, Projects, Skills, and Contact routes with active NavLink indicators, 404 catch-all page, centralized filter state, and dynamic category pill filtering.

Key Endpoints / Routes: Frontend SPA Routes: '/' (Home), '/projects' (Projects), '/skills' (Skills), '/contact' (Contact), '**' (NotFound).

Verification & Test Method: Navigate between routes without full-page reloads. Verify category filtering of projects and dynamic counter badges.

Practical 3: GitHub API Integration & Data Rendering

Directory: Practical_03_GitHub_API | **Localhost URL:** <http://localhost:5173>

Tech Stack: React, Fetch API, GitHub REST API v3, Lucide Icons

Execution & Run Command:

```
npm run p3
```

Module Overview & Architecture:

Asynchronous REST client fetching public repositories and profile data from GitHub. Features debounced username search, animated loading skeleton/spinner, graceful HTTP 403 rate-limit handling, star count sorting, and repository card rendering.

Key Endpoints / Routes: External API: 'https://api.github.com/users/{username}' & '/users/{username}/repos'.

Verification & Test Method: Enter 'Ranakeyur-31525' in search input. Confirm profile avatar, repository cards, stars, forks, and language tags render dynamically.

Practical 4: Node.js & Express RESTful API (In-Memory)

Directory: Practical_04_Express_REST_API | **Localhost URL:** <http://localhost:4004>

Tech Stack: Node.js, Express 4, Custom Middleware

Execution & Run Command:

```
npm run p4
```

Module Overview & Architecture:

Fully compliant RESTful API utilizing an in-memory data store. Implements global request logger middleware recording timestamps, HTTP methods, and route URLs. Includes centralized error handling middleware and an embedded interactive HTML test dashboard at root.

Key Endpoints / Routes: GET / (Web UI or JSON), GET /tasks, GET /tasks/:id, POST /tasks, PUT /tasks/:id, DELETE /tasks/:id.

Verification & Test Method: Open <http://localhost:4004> in browser to test CRUD directly via UI. Or run: `curl http://localhost:4004/tasks`

Practical 5: MongoDB Integration & Mongoose Schema Design

Directory: Practical_05_MongoDB_Mongoose | **Localhost URL:** <http://localhost:4005>

Tech Stack: Node.js, Express, MongoDB, Mongoose 8

Execution & Run Command:

```
npm run p5
```

Module Overview & Architecture:

Production-grade Object Data Modeling (ODM) with Mongoose. Defines Task schema with strict validation rules (required title with minlength 3, priority enum [low, medium, high], default completed false, timestamps). Serves an interactive schema & MongoDB visualizer.

Key Endpoints / Routes: GET / (Web UI or JSON), GET /tasks, GET /tasks/:id, POST /tasks, PUT /tasks/:id, DELETE /tasks/:id.

Verification & Test Method: Open <http://localhost:4005> in browser to view live MongoDB connection state, task documents, and insert records.

Practical 6: Full Stack Integration (React + Express + MongoDB) [MERGED]

Directory: Practical_06_FullStack_App | **Localhost URL:** **Backend:** <http://localhost:5006> | **Frontend:** <http://localhost:5176>

Tech Stack: React, Express, MongoDB, Mongoose, Axios/Fetch, CSS Variables

Execution & Run Command:

```
Backend: npm run p6:backend | Frontend: npm run p6:frontend
```

Module Overview & Architecture:

Complete decoupled full-stack task management application merging React frontend with Express backend and MongoDB database. Features centralized API service layer with error trapping, live connection status pill, real-time metrics banner, and modal workflows.

Key Endpoints / Routes: Backend (Port 5006): GET /, GET /tasks, GET /tasks/:id, POST /tasks, PUT /tasks/:id, DELETE /tasks/:id, GET /api/tasks/stats.

Verification & Test Method: Open frontend on <http://localhost:5176>. Create task, toggle completion, edit priority, and delete task while verifying backend updates on port 5006.

Practical 7: JWT Authentication & Middleware Pipeline

Directory: Practical_07_Auth_Middleware | **Localhost URL:** **Backend:** <http://localhost:5001> | **Frontend:** <http://localhost:5177>

Tech Stack: Node.js, Express, bcryptjs, jsonwebtoken, React

Execution & Run Command:

```
Backend: npm run p7:backend | Frontend: npm run p7:frontend | Test: npm run p7:test
```

Module Overview & Architecture:

End-to-end security pipeline. Password hashing using bcrypt salt rounds before database persistence. JWT issuance on registration/login. Protected route middleware verifying 'Authorization: Bearer ', user extraction, and dedicated /api/auth/me profile endpoint.

Key Endpoints / Routes: POST /api/auth/register, POST /api/auth/login, GET /api/auth/me, GET /api/tasks (Protected), POST /api/tasks (Protected).

Verification & Test Method: Run: npm run p7:test to verify 5 automated assertions (401 blocking, 400 validation, 201 token issuance, 200 /me retrieval).

Practical 8: Performance Optimization & React Lazy Loading

Directory: Practical_08_React_LazyLoading | **Localhost URL:** <http://localhost:5178>

Tech Stack: React 18, React.lazy(), Suspense, Vite Rollup Chunks

Execution & Run Command:

```
npm run p8 | Build Chunks: npm --prefix Practical_08_React_LazyLoading run build
```

Module Overview & Architecture:

Advanced client bundle optimization. Route code-splitting with React.lazy() and Suspense. Component-level on-demand loading for heavy analytics chart. Debounced fallback spinner preventing visual flicker. Rollup manual chunks isolating vendor libraries (-76% bundle reduction).

Key Endpoints / Routes: Vite dev server on port 5178. Chunks: index.js, AnalyticsChart.js, vendor-react.js, vendor-icons.js.

Verification & Test Method: Run: npm --prefix Practical_08_React_LazyLoading run build and observe discrete chunk generation in dist/assets.

Practical 9: In-Memory Caching & Query Optimization

Directory: Practical_09_Caching_QueryOptimization | **Localhost URL:** <http://localhost:5002>

Tech Stack: Node.js, Express, node-cache, MongoDB, Mongoose

Execution & Run Command:

```
Backend: npm run p9 | Benchmark: npm run p9:test
```

Module Overview & Architecture:

Read acceleration using in-memory node-cache with 60-second TTL. Benchmark demonstrates drop from 22.58ms (MongoDB query) to 11.32ms (node-cache hit) -> up to 16.3x speedup. Write operations (POST/PUT/DELETE) automatically invalidate cache keys to ensure zero stale reads.

Key Endpoints / Routes: GET /api/tasks (Cached), POST /api/tasks (Invalidates), GET /api/cache/stats, POST /api/cache/flush.

Verification & Test Method: Run: npm run p9:test. Benchmark runs 3 uncached samples vs 3 cached samples and prints performance speedup factor.

Practical 10: Asynchronous Processing with Event-Driven Architecture

Directory: Practical_10_EventDriven_Async | **Localhost URL:** <http://localhost:5003>

Tech Stack: Node.js, Native EventEmitter, Express, MongoDB

Execution & Run Command:

```
Backend: npm run p10 | Test Ordering: npm run p10:test
```

Module Overview & Architecture:

Decoupled asynchronous execution using native TaskEventEmitter. When tasks are created, HTTP 201 response returns immediately to client while background event listeners handle simulated email/SMS notifications without blocking the event loop.

Key Endpoints / Routes: POST /api/tasks (Triggers task-created), GET /api/tasks/event-logs (Inspects background handler execution).

Verification & Test Method: Run: npm run p10:test. Proves API response timestamp precedes background worker completion timestamp by +1509ms.

Practical 11: Containerization with Docker & Docker Compose

Directory: Practical_11_Docker_Containerization | **Localhost URL:** **Backend:** <http://localhost:5011> | **Frontend:** <http://localhost:5181>

Tech Stack: Docker, Docker Compose, Multi-stage Dockerfile, Nginx Alpine, Node 18, MongoDB 6.0

Execution & Run Command:

```
Local: npm run p11:backend & p11:frontend | Compose: docker-compose -f  
Practical_11_Docker_Containerization/docker-compose.yml up
```

Module Overview & Architecture:

Three-tier container orchestration. Multi-stage frontend Dockerfile reduces image size from 468 MB to 22.4 MB on Nginx Alpine. Services communicate via internal bridge network 'app-network' using container DNS. Named volume 'mongo-data' ensures database durability.

Key Endpoints / Routes: Docker Services: frontend (port 5173:80), backend (port 5000:5000), mongodb (port 27017:27017).

Verification & Test Method: Validate compose config: docker-compose -f Practical_11_Docker_Containerization/docker-compose.yml config

Practical 12: CI/CD Pipeline with GitHub Actions & Automated Testing

Directory: Practical_12_CI_CD_GitHub_Actions | **Localhost URL:** <http://localhost:5012>

Tech Stack: GitHub Actions, Node test runner, ESLint, Express

Execution & Run Command:

```
Server UI: npm run p12 | Automated Tests: npm run p12:test
```

Module Overview & Architecture:

Automated continuous integration pipeline (.github/workflows/ci.yml). Triggers on push and pull requests to main branch. Runs linting and 5 assertion test cases. Demonstrates automated pipeline halting (exit code 1) on broken tests vs passing checks (exit code 0).

Key Endpoints / Routes: Server (Port 5012): GET / (CI Visualizer Dashboard), GET /api/run-tests, GET /api/run-broken-test.

Verification & Test Method: Run: npm run p12:test to verify 5/5 assertions pass. Open <http://localhost:5012> to test pass/fail modes interactively.

Practical 13: AI API Integration with Google Gemini Proxy

Directory: Practical_13_AI_Integration | **Localhost URL:** **Backend:** <http://localhost:5004> | **Frontend:** <http://localhost:5183>

Tech Stack: Google Gemini 1.5 Flash API, Express, Rate Limiter, React

Execution & Run Command:

```
Backend: npm run p13:backend | Frontend: npm run p13:frontend
```

Module Overview & Architecture:

Secure backend proxy architecture protecting Gemini API key in server-side .env. In-memory rate limiting middleware enforcing 1 request/minute cooldown (HTTP 429). Intelligent graceful degradation fallback generating valid suggestions when unkeyed or offline.

Key Endpoints / Routes: POST /api/ai/generate-description (Rate limited, returns JSON with description, priority, hours, and disclaimer).

Verification & Test Method: Send POST request to <http://localhost:5004/api/ai/generate-description> with body {'title': 'Deploy Kubernetes Cluster'}. Verify response & disclaimer.

Master Showcase Portal (Combined All-in-One Application)

Directory: combined-app/ | **Frontend:** <http://localhost:5180> | **Backend:** <http://localhost:5000>

Run Commands:

```
Backend: npm run dev:backend Frontend: npm run dev:frontend
```

Overview:

The Master Showcase portal unifies all 13 course practicals into a single responsive, dark/light themed web application. Features dedicated interactive tab pages for each practical (/p1 through /p13), live MongoDB status indicator, interactive task CRUD workflows, cache benchmark visualizers, event stream monitors, and AI suggestion generation.

Verification & Academic Deliverables Checklist

All 13 practicals have been implemented, tested, and validated according to the CHARUSAT AWDF (ITUE301) course syllabus:

- ✓ **13 Dedicated Project Directories:** Clean symmetrical naming (Practical_01_ to Practical_13_).
- ✓ **13 Standardized 10-Page Lab Report PDFs:** Located in Lab_Reports_PDFs/ and within each folder.
- ✓ **Conflict-Free Port Allocation:** Every practical server runs on its own dedicated port.
- ✓ **Interactive Web Test Dashboards:** Pure backend practicals offer responsive browser test UIs.
- ✓ **GitHub Repository:** Source code and reports synchronized with <https://github.com/Ranakeyur-31525/awdf>.

Student Certification:

I hereby confirm that all 13 practical implementations, automated test suites, and documentation presented in this handbook represent my original coursework for the Advance Web Development Framework (ITUE301) laboratory curriculum.

Keyur Rana (Student ID: D25DCE176)

Department of Information Technology, CSPIT, CHARUSAT University